Nomikos: Advanced Programming report

Krrish Agarwalla and Astha Gupta

Live at `nomikos.app`, code at `github.com/kkkamur07/nomikos`.

Nomikos is a web platform for handwritten text recognition of Byzantine Greek, Armenian, Syriac and Coptic manuscripts. A researcher uploads page images and runs a segmenter, which finds the lines on a page, and a recogniser, which reads each line. They fix the output in a browser editor and export PAGE XML (a standard format for page layout and text), docx, PDF or plain text.

We built Nomikos around two goals.

1. **Researchers verify, they do not transcribe.** The models and the platform do the heavy lifting: they find the lines on a page and read them. The researcher checks the result and fixes what is wrong. Our target is to make transcription ten times faster than typing it by hand. Every fix is stored apart from the model's guess (Section 4), so it becomes training data for the next model.
2. **Models in the researcher's workflow, without the setup.** Running a research model usually means a GPU, a Python environment and the authors' training code. On Nomikos a researcher uploads pages in the browser, picks a model and gets the output, with nothing to install. Jobs run on our hosted runner, or on the researcher's own computer, where the compute costs us nothing.

Existing HTR platforms did not fit these goals. eScriptorium is open source but built around one engine, kraken, and someone has to host an instance. Transkribus lets users train and apply their own models, but only inside its own cloud, where recognition is metered in credits.

1 System overview

Figure 1 shows two paths.

On the request path, the browser loads the editor from Vercel once and from then on sends every call to the FastAPI API, the only backend service that accepts incoming HTTP. The API holds all business logic. It reads and writes Postgres, keeps page images in a media directory on the host, and streams job updates back to the browser as server-sent events (SSE), a one-way stream over an ordinary HTTP connection.

On the work path, models run in the *inference runner*, a small Python program published on PyPI. The researcher chooses where a job runs: locally, on a runner installed on their own computer with one command, or in the cloud, on the runner we host, which needs no install. Both are the same program. A runner asks the API for a job, downloads the page image, runs the model and posts the result back. GitHub Actions publishes each runner release to PyPI, and the runner fetches model weights from the Hugging Face Hub. A separate platform worker handles jobs that need no model, such as building exports, and reads the same media directory without write access.

All stateful parts sit on one Coolify host: the API container, the Postgres container, the media directory and the platform worker. Postgres is backed up off the host to Cloudflare. The hosted runner lives on a separate machine as systemd units.

Frontend on Vercel, backend on Coolify. Vercel builds and serves Next.js natively and caches it on a global edge network, so the editor loads fast. The backend needs what serverless platforms handle poorly: a long-running worker, a direct database connection, a disk for images and connections that stay open for the SSE streams (Section 2). Coolify deploys our containers from Git on our own server, and we take care of that server, its releases and its backups.

The frontend runs entirely in the browser. TanStack Query caches every API response for 30 seconds and refetches when the tab regains focus or the network comes back. In the editor, a line's text is saved when the line loses focus, edits can be undone, and saved states of a page go into a history the user can restore from. While a job runs, all components that care about it share one event stream for that job, and if no event arrives for 12 seconds the browser falls back to polling.

We chose tools with broad support that we already knew (Table 1).

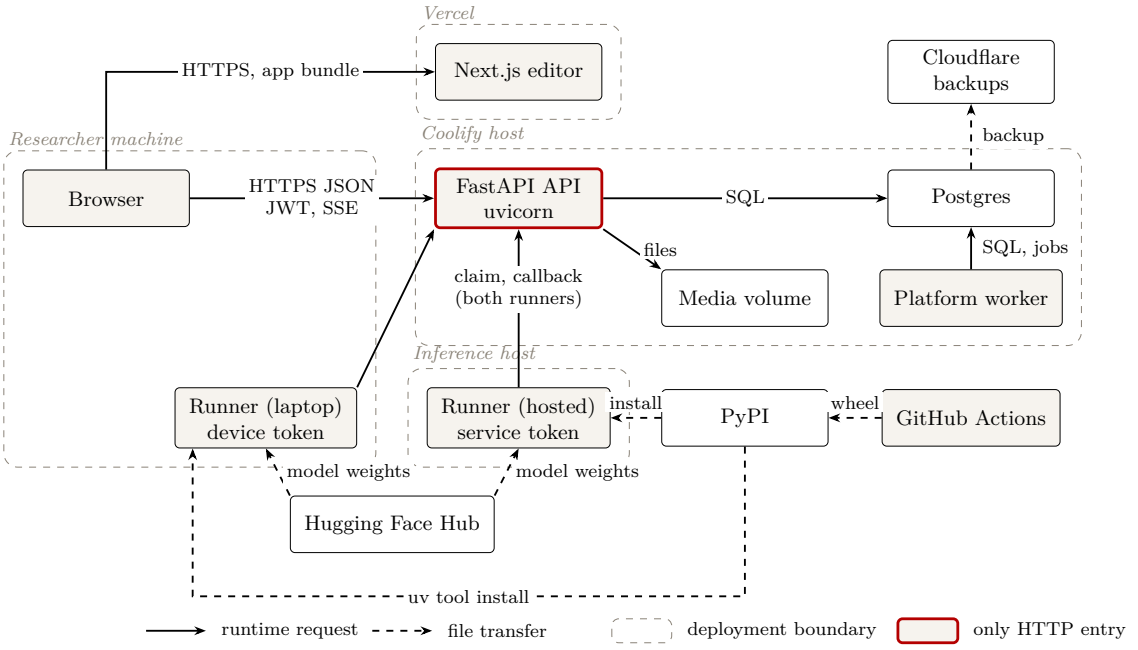


Figure 1: Containers and where they run. The browser loads the editor from Vercel and then talks to the API directly. Both runners are the same PyPI package and only make outbound calls.

Layer	Choice	Functionality
Frontend	Next.js 16, React 19, TypeScript	The web app researchers use: upload pages, check and correct the model output, export the result
API	FastAPI, SQLAlchemy 2 (async)	The server behind the app: answers every request, checks who may see and change what, and holds all business logic
Database	PostgreSQL 16, Alembic	Stores all data and also serves as the job queue; Alembic keeps the revision history of the schema
Inference	ONNX Runtime on CPU	Runs the machine learning models that find lines and read text, on an ordinary computer without a GPU
Models	Hugging Face Hub	Stores the trained model files, each pinned to an exact version; our models and datasets are public there
Hosting	Vercel, Coolify	Puts the system online: Vercel serves the web app, Coolify runs the server and database on our machine
Backups	Cloudflare	Keeps copies of the data away from our server
CI	GitHub Actions	Checks and tests every change automatically and publishes new releases

Table 1: The stack. The Python inference runner is published on PyPI as `nomikos-inference`.

2 Job flow

A job is one page and one model, and Figure 2 follows it. Three POST requests drive it, and each one is a single database transaction, so a crash leaves the job in a known state. The browser's POST inserts a `pending` row and returns at once. A runner's claim POST locks one pending row, skipping rows other runners hold, and marks it `waiting`. The runner fetches the image through a signed URL, runs the model locally and sends the result in the completion POST. That handler locks the job, sets `callback_claimed_at` so a retried callback cannot merge twice, writes the run and the lines, marks the job `done` and fires `NOTIFY`, which wakes every process listening on that Postgres channel. Each API process keeps one `LISTEN` connection that feeds all its open streams, and it pushes the result to the browser over the job's SSE stream at `/jobs/{id}/events`. If the stream is blocked, the browser polls `GET /jobs/{id}`. A runner reports a heartbeat while it works. If the heartbeats stop, its lease expires and the job goes back to `pending`; after five expired leases the job is marked failed.

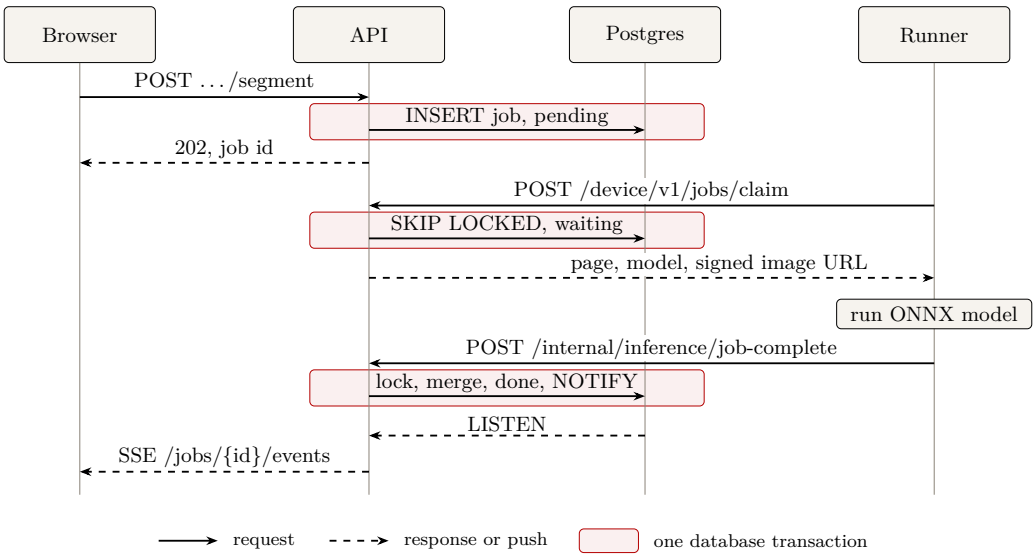


Figure 2: One segmentation job. Each of the three POST requests is a single transaction; the browser hears about the result through `NOTIFY` and server-sent events.

3 Design decisions

One process, six contexts. The API is one process split into six bounded contexts (users, project, document, annotation, ml, jobs), each with `api`, `application`, `domain` and `infrastructure` layers. Microservices would add network hops, several databases and distributed transactions. A monolith gives one deploy and one transaction per request, and one schema whose revisions Alembic records. The price is that boundaries are only conventions: document, annotation, ml and jobs import each other in both directions.

Postgres is the queue. The `jobs` table is the only queue. We did not add Redis or RabbitMQ, so a job and the data it writes commit in the same transaction and nothing can drift between two stores. Runners pull work with `SELECT ... FOR UPDATE SKIP LOCKED`, which lets many workers claim rows without blocking each other. The cost is that Postgres carries the queue load on top of every other request.

Runners connect outward. Only the token tells the two runners apart (Figure 6). A device token claims its owner's local jobs, a service token claims cloud jobs. This splits the compute bill. A local job runs on the researcher's own machine and costs us nothing; a cloud job runs on our host, which is the only inference we pay for. The hosted runner serves anyone without a capable laptop or who does not want to install anything. Because the runner always opens the connection, it works behind a university firewall without port forwarding, and the server never needs to know where runners are. The alternative, a server that pushes work to a listening inference service, needs open ports, a registry of worker addresses and a second code path. The cost is that runners stop whenever the API is down.

ONNX Runtime serves, PyTorch only exports. The runner lives on researchers' laptops, some with only 8 GB of memory, so we chose ONNX Runtime because it is light. Figure 3 compares it with PyTorch for serving.

One CPU build installs on every platform, so nobody needs a special flag to keep GPU libraries off their laptop. PyTorch is still part of the project, but only to export each trained model to an ONNX file (Figure 4). Parity tests check that the exported model gives the same output as the original, and a registry file shipped with the runner stores each model's preprocessing settings.

Adding a model follows the same path: export it and pass the parity test, publish it to the Hub, add an entry with its revision and file hash to the registry, insert a row in the model catalog table so it appears in the editor's model picker, deploy the API, and release a new runner so runners carry the entry. Each runner downloads the weights on first use.

The API contract is generated. A script exports the OpenAPI description of the FastAPI

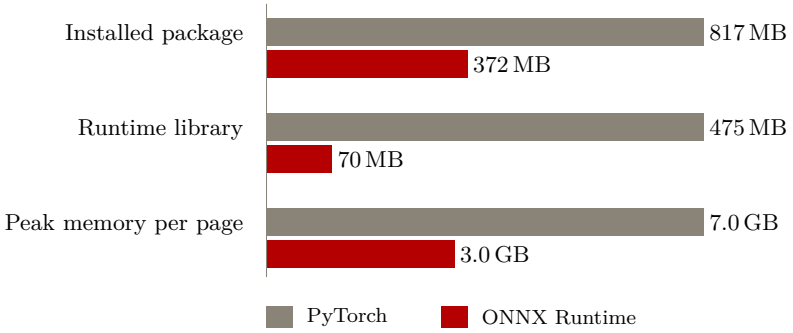


Figure 3: Serving with PyTorch against ONNX Runtime, bars scaled to PyTorch. The installed package is 54% smaller. Measured on macOS arm64; numbers from our architecture decision records.

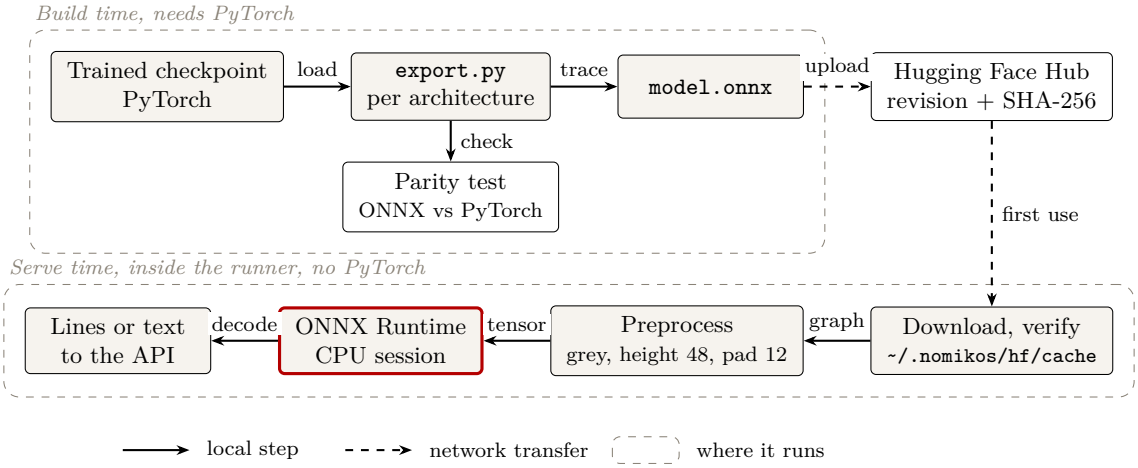


Figure 4: From a trained model to a served one. The registry pins each model to a Hub revision and a file hash, so every runner loads the same bytes.

app (115 operations), and openapi-typescript turns it into TypeScript types for the frontend. A pre-commit hook regenerates both, and CI fails if they drift. If the backend drops a field, the frontend stops compiling, so the two sides cannot go out of sync on `main`.

4 Data model

Model output is never overwritten. Each segmentation run is stored once in `segmentation_runs` as JSON with the model name and version, and never updated. Human edits go to the live lines and to a separate `ground_truth` transcription layer. Comparing the two shows exactly what a person fixed, including lines they deleted, and that difference is the training data. Overwriting the model output in place would save space but lose that record, so we keep every run and accept that storage only grows.

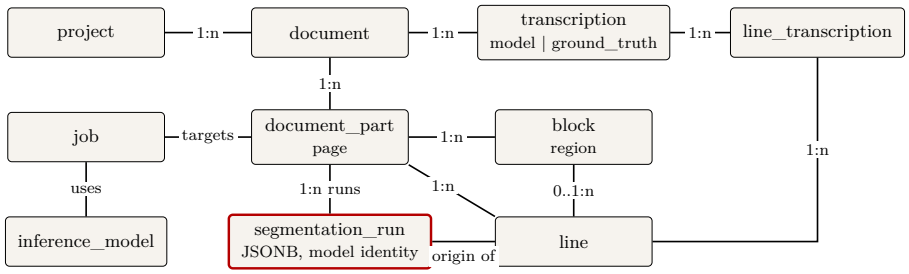


Figure 5: Core entities; users, sharing and history tables are left out. The outlined entity is written once and never updated.

A project holds documents, a document holds pages, and a page holds regions and lines (Figure 5). Text does not live on the line. It lives in `line_transcription`, one row per line and per transcription

layer, so the model's text and the human's text sit side by side. Every model-proposed line points back to the segmentation run that proposed it, and the run keeps the model identity as text, so the history survives even if the job or the catalog row is deleted. A job row carries its own claim state (who holds it, how often it was claimed, when it last reported), which is all the queue needs.

5 Security

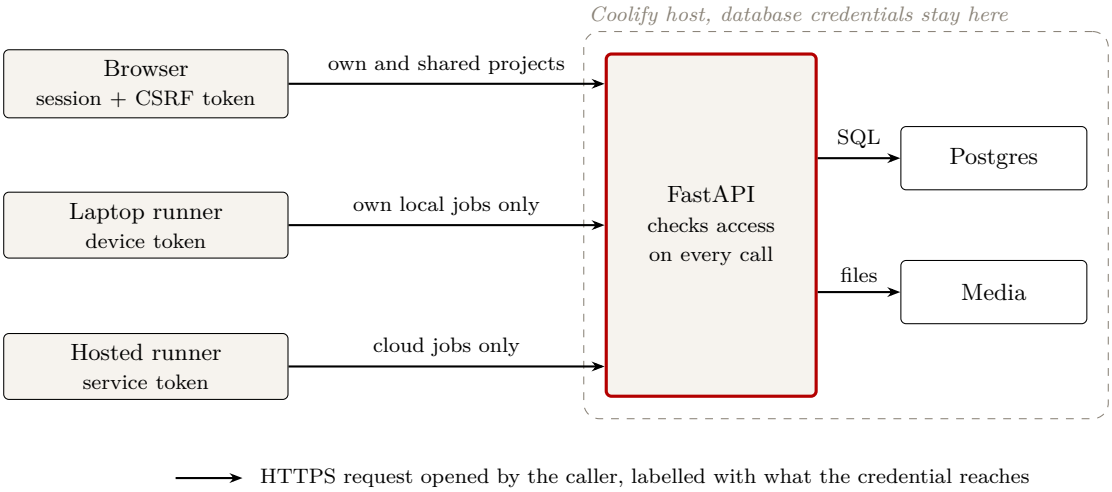


Figure 6: Three credentials, one entry point. Each caller gets only the rights its token carries, and runners only ever open connections to the API, so no inbound port is needed on a laptop or behind a university firewall. Nothing outside the host holds a database password.

Figure 6 shows who can reach what. The browser never talks to Postgres or storage directly, and images come through short-lived signed URLs. We chose not to use Postgres row-level security, so FastAPI checks access before every query. All authorisation lives in one layer, which is easy to audit but means every new route has to remember the check. FastAPI also owns authentication: password hashes, JWT access tokens with rotating sessions, CSRF checks and rate-limited logins. A laptop runner never sees a password. It pairs once through a flow modelled on the OAuth device authorization grant (RFC 8628): the runner opens the browser, the signed-in user approves the device, and the runner receives a device token that it sends in its own header. That token can claim only its owner's local jobs.

The completion callback is narrowed in the same way. The API records which runner holds each job in `jobs.claimed_by` and accepts a result only from that runner, so one paired laptop cannot complete or fail anyone else's work. Runners get only inference and callback credentials, never database or JWT secrets. Before loading a model, a runner checks the file against the SHA-256 hash pinned in the registry.

6 Usage and quality

Table 2 gives the numbers from the admin dashboard for the 90 days up to 30 October 2026. They show that the platform is used, not yet how much faster transcription has become: we have not measured the ten-times target.

CI on GitHub Actions runs type checks, linting and tests for both frontend and backend, integration tests against a real Postgres 16, a nightly run on real model weights and the API contract check. Secret scanning, dependency audits and container scans run alongside, and the runner is released to PyPI through trusted publishing. The backend has 889 unit tests and 298 integration tests, of which 170 need a running Postgres. A local run of the unit tests alone covers 56 % of statements and branches in the backend and the runner. An admin dashboard shows users, jobs per model with success rate and median duration, request timings per route, and the failure reports and feedback that users send.

Measure	Value
Accounts (four of them test or service accounts)	18
Jobs run	1,070
transcriptions	876
segmentations	194
Median transcription time per page, by model	2.5 to 3.3 s
Median segmentation time per page, current segmenters	14 to 36 s

Table 2: Usage over 90 days. Times run from job creation to completion.

7 Limits and next steps

Two weaknesses matter today:

- **Schema changes by hand.** Alembic holds the revision history, but the production container never runs it. Before Coolify deploys, a release script applies dated SQL by hand and checks the recorded Alembic version; running migrations automatically would be safer.
- **One host.** The API, Postgres and the media directory share one Coolify host, so it is a single point of failure. Backups leave the host, so the data survives a lost machine, but the service does not survive an outage. The first step is to give Postgres its own host or a managed service.